

Build Your First DCGM API Clients

This tutorial follows [Getting Started for Application Developers](#). Choose the host-engine deployment that matches the application's ownership model before implementing either client.

The development package installs public headers, the `DCGM::dcgm` CMake target, and SDK samples. The examples on this page use the C interface and perform one operation: list the physical GPUs that the host engine reports as supported. Keeping the operation the same makes the two host-engine lifecycles easy to compare.

CMake and Build-System Considerations

Install the runtime and development packages described in [Start and Verify DCGM](#). Download these three files into one directory:

- [↓ CMakeLists.txt](#)
- [↓ standalone.c](#)
- [↓ embedded.c](#)

The build uses the same CMake package target as the installed SDK samples:

```
# SPDX-License-Identifier: Apache-2.0

cmake_minimum_required(VERSION 3.14)
project(dcgmm_minimal_examples LANGUAGES C)

find_package(DCGM 4.0 REQUIRED CONFIG)

add_executable(dcgmm-standalone standalone.c)
target_compile_features(dcgmm-standalone PRIVATE c_std_11)
target_link_libraries(dcgmm-standalone PRIVATE DCGM::dcgm)

add_executable(dcgmm-embedded embedded.c)
target_compile_features(dcgmm-embedded PRIVATE c_std_11)
target_link_libraries(dcgmm-embedded PRIVATE DCGM::dcgm)
```

`find_package(DCGM 4.0 REQUIRED CONFIG)` loads the package configuration installed by DCGM instead of relying on a third-party CMake find module. The version is a minimum compatible API baseline: the two clients use APIs available in DCGM 4.0. Raise it to the oldest release that supplies every API used by the application. Use `EXACT` only when the application intentionally rejects later compatible releases.

Link to `DCGM::dcgm` instead of hard-coding an include directory, library directory, or architecture-specific library filename. The imported target carries the DCGM include path and shared-library link information. Use a `PRIVATE` dependency for an executable or for a library whose public headers do

not expose DCGM types. Use `PUBLIC` when consumers must compile public headers that include DCGM headers or mention DCGM types.

The normal imported target links the application to the `libdcgm.so.4` runtime ABI. The unversioned `libdcgm.so` linker name and headers are build dependencies; the versioned library is a deployment dependency. Declare the DCGM 4 runtime package in the application's packaging metadata instead of copying a library from the build host. If the forwarding stub is selected instead, the build must find `libdcgm_stub.a` and link `${CMAKE_DL_LIBS}`; the target system still needs `libdcgm.so.4`. See [C/C++ API](#) for the two linking models.

For a nonstandard installation prefix, pass `CMAKE_PREFIX_PATH` when configuring rather than embedding a machine-specific path in `CMakeLists.txt`. For cross-compilation, install the target-architecture development package in the sysroot and use a CMake toolchain file that keeps package discovery inside that sysroot. Finding the build host's `DCGMConfig.cmake` can otherwise select headers and a library for the wrong architecture.

Keep compile-and-link tests separate from runtime integration tests. Building these clients does not require a GPU. A standalone runtime test additionally needs a reachable host engine; an embedded runtime test needs the DCGM runtime and the device and operating-system permissions required by the APIs under test.

Build the Examples

Configure the project from the directory containing the downloaded files:

```
$ cmake -S . -B build
```

Build both clients:

```
$ cmake --build build
```

The development package normally places `DCGMConfig.cmake` in a standard CMake search location, so no additional discovery option is needed for a system package installation.

Standalone Example: Connect to a Shared Engine

This client expects `nv-hostengine` to be running on the default loopback endpoint, `127.0.0.1:5555`. The packaged `nvidia-dcgm` service provides that endpoint with its default configuration. The application initializes its client library, connects, uses the returned handle, disconnects, and shuts down its client library:

```

1 // SPDX-License-Identifier: Apache-2.0
2
3 #include <dcgm_agent.h>
4
5 #include <stdio.h>
6
7 static int list_supported_gpus(dcgMHandle_t handle)
8 {
9     unsigned int gpuIds[DCGM_MAX_NUM_DEVICES] = { 0 };
10    int count = 0;
11    dcgmReturn_t result = dcgmGetAllSupportedDevices(handle, gpuIds, &count);
12
13    if (result != DCGM_ST_OK)
14    {
15        fprintf(stderr, "dcgmGetAllSupportedDevices failed: %s\n", errorString(result));
16        return 1;
17    }
18
19    if (count == 0)
20    {
21        puts("DCGM reports no supported GPUs.");
22        return 0;
23    }
24
25    printf("DCGM reports %d supported GPU%s:", count, count == 1 ? "" : "s");
26    for (int i = 0; i < count; ++i)
27    {
28        printf(" %u", gpuIds[i]);
29    }
30    putchar('\n');
31    return 0;
32 }
33
34 int main(void)
35 {
36    dcgmHandle_t handle = 0;
37    dcgmReturn_t result = dcgmInit();
38
39    if (result != DCGM_ST_OK)
40    {
41        fprintf(stderr, "dcgmInit failed: %s\n", errorString(result));
42        return 1;
43    }
44
45    result = dcgmConnect("127.0.0.1", &handle);
46    if (result != DCGM_ST_OK)
47    {
48        fprintf(stderr, "dcgmConnect failed: %s\n", errorString(result));
49        result = dcgmShutdown();
50        if (result != DCGM_ST_OK)
51        {
52            fprintf(stderr, "dcgmShutdown failed: %s\n", errorString(result));
53        }
54        return 1;
55    }
56
57    int exitCode = list_supported_gpus(handle);
58
59    result = dcgmDisconnect(handle);
60    if (result != DCGM_ST_OK)
61    {
62        fprintf(stderr, "dcgmDisconnect failed: %s\n", errorString(result));
63        exitCode = 1;

```

```

64     }
65
66     result = dcgmShutdown();
67     if (result != DCGM_ST_OK)
68     {
69         fprintf(stderr, "dcgmShutdown failed: %s\n", errorString(result));
70         exitCode = 1;
71     }
72
73     return exitCode;
74 }

```

Run the client:

```
$ ./build/dcgm-standalone
```

For example, a system with two supported physical GPUs might print:

```
DCGM reports 2 supported GPUs: 0 1
```

`dcgmDisconnect()` releases this client's connection. It does not stop the standalone host engine or discard state that other connected clients still use. For a non-default TCP port or another transport, make the endpoint part of application configuration and use the appropriate connection API; see [TRANSPORT AND CLIENT ADDRESSES](#).

Embedded Example: Own the Engine

This client starts the host engine inside the application process. It does not connect to the separately running `nv-hostengine` service. The application initializes the library, starts an embedded engine, uses its handle, stops the engine, and shuts down the library:

```

1 // SPDX-License-Identifier: Apache-2.0
2
3 #include <dcgm_agent.h>
4
5 #include <stdio.h>
6
7 static int list_supported_gpus(dcgMHandle_t handle)
8 {
9     unsigned int gpuIds[DCGM_MAX_NUM_DEVICES] = { 0 };
10    int count = 0;
11    dcgmReturn_t result = dcgmGetAllSupportedDevices(handle, gpuIds, &count);
12
13    if (result != DCGM_ST_OK)
14    {
15        fprintf(stderr, "dcgmGetAllSupportedDevices failed: %s\n", errorString(result));
16        return 1;
17    }
18
19    if (count == 0)
20    {
21        puts("DCGM reports no supported GPUs.");
22        return 0;
23    }
24
25    printf("DCGM reports %d supported GPU%s:", count, count == 1 ? "" : "s");
26    for (int i = 0; i < count; ++i)
27    {
28        printf(" %u", gpuIds[i]);
29    }
30    putchar('\n');
31    return 0;
32 }
33
34 int main(void)
35 {
36    dcgmHandle_t handle = 0;
37    dcgmReturn_t result = dcgmInit();
38
39    if (result != DCGM_ST_OK)
40    {
41        fprintf(stderr, "dcgmInit failed: %s\n", errorString(result));
42        return 1;
43    }
44
45    result = dcgmStartEmbedded(DCGM_OPERATION_MODE_AUTO, &handle);
46    if (result != DCGM_ST_OK)
47    {
48        fprintf(stderr, "dcgmStartEmbedded failed: %s\n", errorString(result));
49        result = dcgmShutdown();
50        if (result != DCGM_ST_OK)
51        {
52            fprintf(stderr, "dcgmShutdown failed: %s\n", errorString(result));
53        }
54        return 1;
55    }
56
57    int exitCode = list_supported_gpus(handle);
58
59    result = dcgmStopEmbedded(handle);
60    if (result != DCGM_ST_OK)
61    {
62        fprintf(stderr, "dcgmStopEmbedded failed: %s\n", errorString(result));
63        exitCode = 1;

```

```

64     }
65
66     result = dcgmShutdown();
67     if (result != DCGM_ST_OK)
68     {
69         fprintf(stderr, "dcgmShutdown failed: %s\n", errorString(result));
70         exitCode = 1;
71     }
72
73     return exitCode;
74 }

```

Run the client:

```
$ ./build/dcgm-embedded
```

The example selects `DCGM_OPERATION_MODE_AUTO` so DCGM performs background updates when later code adds watches or policy. In manual mode, merely calling the read APIs is not enough; the embedding application must schedule the required update and policy-trigger calls.

`dcgmStopEmbedded()` destroys this embedded engine and its runtime state. The application process also supplies the credentials and environment under which the engine and supported child processes operate. Treat embedding as ownership of a long-lived node service, not as a helper to start once per request.

Compare the Lifecycles

Minimal client lifecycle

Phase	Standalone	Embedded
Initialize the library	<code>dcgmInit()</code>	<code>dcgmInit()</code>
Obtain a handle	<code>dcgmConnect()</code> to an existing engine	<code>dcgmStartEmbedded()</code> to create the engine
Call DCGM APIs	Use the connection handle	Use the embedded-engine handle
Release the handle	<code>dcgmDisconnect()</code>	<code>dcgmStopEmbedded()</code>
Shut down the library	<code>dcgmShutdown()</code>	<code>dcgmShutdown()</code>
Engine after client cleanup	Continues under its service owner	No longer exists

Both examples treat a zero GPU count as a valid query result. They also check the return status of every operation. `dcgmGetAllSupportedDevices()` reports supported physical GPU IDs only; it is not

complete inventory for MIG, NVSwitch, CPU, link, or adapter entities. Entity IDs are scoped to the running host engine, so query inventory again after an engine restart.

As the client grows, initialize every versioned structure to zero and set its `version` member before use. For field values, inspect both the API return and each value’s status and timestamp. A successful API call can still contain an unavailable, unsupported, permission-denied, or not-yet-sampled value.

Continue with the Installed SDK Samples

The development package installs more complete samples under `/usr/src/datacenter-gpu-manager-4/sdk_samples/`. After implementing the minimal lifecycle above, select the sample that matches the next application feature:

Installed SDK sample guide

Sample	What to study next
<code>c_src/configuration_sample</code>	Discover GPUs; create a group; get, set, and enforce configuration; and inspect per-GPU errors through a status handle.
<code>c_src/field_value_sample</code>	Create entity and field groups, install field watches, and retrieve the latest values and retained history.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.